Laboratório de Concorrência · Aula 10

Johan Gabriel da Silva dos Santos · RA 24190316-2 · Turma B
Gerado em 16/10/2026, 15:10:39 · tempo registrado no cronômetro: 62:56

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma tarefa precisa esperar outra terminar para continuar. Um exemplo do dia a dia é esperar um colega terminar de preparar a comida antes de começar a servir. Já a sincronização de competição acontece quando duas tarefas querem utilizar o mesmo recurso ao mesmo tempo, sendo necessário organizar o uso para evitar conflitos. Um exemplo é duas pessoas querendo usar o mesmo caixa eletrônico ao mesmo tempo, sendo necessário esperar a vez.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Eu espero que o programa imprima 2.000.000, pois cada uma das duas threads soma 1.000.000 ao contador

1ª execução (n = 1.000.000)

1726643

2ª execução (n = 1.000.000)

1289544

3ª execução (n = 1.000.000)

1269281

Saída do programa

```
1º execução:
esperado: 2000000
obtido:   1726643

2º execução:
```

```
esperado: 2000000
obtido:   1289544

3° execução:
esperado: 2000000
obtido:   1269281
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. Os valores obtidos foram menores que o esperado e não foram iguais nas três execuções. Isso acontece devido à condição de corrida: as duas threads acessam e modificam o mesmo contador ao mesmo tempo, fazendo com que algumas somas sejam perdidas. Como a execução das threads varia, os resultados também mudam.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

1. Ler: A Thread 1 e a Thread 2 leem o mesmo valor do contador, por exemplo, 10.
2. Somar: As duas threads somam 1 ao valor lido, chegando a 11.
3. Gravar: As duas gravam o valor 11 no contador.

O correto seria o contador chegar a 12, mas como as duas gravaram 11, uma das somas foi perdida. Isso caracteriza uma condição de corrida.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, os resultados podem ficar mais próximos do esperado ou até serem iguais a 2000, pois há menos operações e menor chance de ocorrer uma condição de corrida. Esse erro é perigoso em sistemas reais porque pode passar despercebido nos testes, mas causar resultados incorretos em produção, como erros em saldos bancários, estoques ou transações financeiras.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

```
141 ms
AAABBB → 8
BBBAAA → 7
```

```
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens de execução, que produzem 4 resultados diferentes: 4, 6, 7 e 8. Apenas 1 ordem produz o resultado correto (8), que é AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Escolhi a ordem ABABBA. A tarefa A lê o valor 3, enquanto B também lê 3. A calcula 3 + 1 = 4 e B calcula 3 × 2 = 6. B grava 6 em TOTAL, mas depois A grava 4, sobrescrevendo o valor anterior. Assim, o resultado final é 4. A operação de B foi perdida porque a gravação de A sobrescreveu o resultado de B.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão sem sincronização será a mais rápida, pois não possui mecanismos de controle entre as threads. Porém, ela pode apresentar resultados incorretos devido à condição de corrida. As versões com synchronized e AtomicInteger devem produzir o resultado correto, pois garantem a atualização segura do contador compartilhado.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1370800 em 4 ms
synchronized:      2000000 em 144 ms
AtomicInteger:     2000000 em 24 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto, enquanto a versão sem sincronização pode apresentar um resultado incorreto devido à condição de corrida. As versões sincronizadas costumam ser mais lentas porque precisam controlar o acesso ao contador compartilhado, evitando que as threads alterem o valor ao mesmo tempo.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo garante que cada operação individual seja atômica, mas não garante que a transferência inteira seja executada como uma única operação. Seria necessário sincronizar a retirada e o depósito em conjunto, garantindo que as duas operações sejam concluídas corretamente e evitando inconsistências nos saldos.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens "A terminou" e "B terminou" provavelmente não vão aparecer, pois a thread A bloqueia o recurso X e espera pelo Y, enquanto a thread B bloqueia o recurso Y e espera pelo X. Isso causa um impasse (deadlock), em que nenhuma das threads consegue continuar.

Saídas (antes e depois da correção)

```
Antes da correção:
IMPASSE: 2 threads presas

Depois da correção:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A ficou com o recurso X e esperava pelo recurso Y, que estava bloqueado pela thread B. Ao mesmo tempo, a thread B ficou com o recurso Y e esperava pelo recurso X, que estava bloqueado pela thread A. Assim, nenhuma conseguia continuar, causando um impasse (deadlock).

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que uma thread fique esperando por um recurso enquanto mantém outro bloqueado em uma ordem inversa. Isso elimina essa situação de impasse. Um exemplo real seria um sistema bancário em que duas transferências simultâneas acessam contas diferentes em ordens opostas, podendo ficar bloqueadas uma esperando pela outra.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, espero que leve aproximadamente 200 ms, pois cada trabalhador pode executar 2 tarefas de 100 ms. Com 1 trabalhador, espero aproximadamente 600 ms, pois ele fará as 6 tarefas em sequência. Com 6 trabalhadores, espero aproximadamente 100 ms, pois cada trabalhador poderá executar uma tarefa ao mesmo tempo.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 2 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms


trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

trabalhador 2 fez a tarefa 3
trabalhador 5 fez a tarefa 6
```

```
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o programa levou 600 ms porque as seis tarefas de 100 ms foram executadas em sequência. Com 3 trabalhadores, levou 200 ms, pois cada um executou duas tarefas. Com 6 trabalhadores, levou 100 ms, porque todos puderam executar uma tarefa ao mesmo tempo.
O programa não precisa de synchronized nem de travas porque as tarefas são distribuídas por canais (channels), que permitem a comunicação entre as goroutines sem que elas precisem compartilhar e modificar diretamente uma mesma variável.

Saídas (antes e depois da troca)

```
1° Execução:
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2


2° Execução:
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não possui espaço para armazenar mensagens. Ao enviar o valor 1, o programa fica esperando que alguém receba a mensagem, mas a instrução de recebimento só seria executada depois do envio. Isso causa um deadlock. Ao usar `make(chan int, 1)`, o canal passa a ter espaço para armazenar uma mensagem. Assim, o valor 1 é armazenado sem bloquear o envio e, em seguida, é recebido e impresso normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa apresentará um erro de compilação, pois o Rust não permite que uma thread altere diretamente uma variável pertencente à função main sem garantir a segurança do compartilhamento.

Saídas (o erro e o resultado com move)

```
1ª execução — erro de compilação:
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
Some errors have detailed explanations: E0373, E0502.

2ª execução — com move:
Warnings: value captured by `contador` is never read
Warnings: value assigned to `contador` is never read
Resultado impresso: 0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move transfere a propriedade do contador para a thread, fazendo com que ela trabalhe com sua própria variável, sem atualizar o valor original de main. Por isso, o valor impresso continua sendo 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro da condição de corrida foi percebido durante a execução do programa, quando o contador apresentou resultados incorretos. Em Rust, o compilador detectou o problema antes da execução, impedindo o compartilhamento inseguro da variável. A vantagem do Rust é evitar muitos erros de concorrência antes de o programa rodar. O custo é que o programador precisa aprender e utilizar mecanismos como Arc e Mutex, tornando o código um pouco mais complexo.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem será A, D, C e B. As letras A e D são impressas imediatamente. A letra C é impressa em seguida, pois as promessas são executadas antes dos timers. Depois, B é impressa. A letra E aparece por último, pois o laço ocupado bloqueia a única thread por aproximadamente 300 ms, atrasando a execução dos timers.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 301 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C, B e E. A letra B, mesmo com tempo 0, saiu depois de D e C porque o setTimeout coloca a tarefa na fila de timers, enquanto a promessa que imprime C é executada antes, na fila de microtarefas. Além disso, D é impresso antes de o código atual terminar.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms rodou depois de aproximadamente 301 ms porque o laço ocupado(300) bloqueou a única thread por cerca de 300 ms. Isso significa que um servidor Node.js que realiza um cálculo pesado pode atrasar o atendimento de muitos usuários ao mesmo tempo, prejudicando o tempo de resposta das requisições.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, permitindo que as duas pessoas comprem o mesmo ingresso, causando uma venda duplicada. Para evitar isso, é necessário sincronizar a operação de compra no servidor e garantir que a verificação de disponibilidade e a reserva do ingresso sejam feitas como uma única operação atômica

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que duas tarefas executadas ao mesmo tempo podem gerar resultados incorretos e que cada linguagem possui formas diferentes de evitar esse problema.